

CMPS 6610 Problem Set 03

Answers

Name: _____ Rob Hartley

Place all written answers from `problemset-03.md` here for easier grading.

- **1b.**

$$W(n) = W(n-1) + 1$$

Level 1

$$W(n) = (W(n-1-1) + 1) + 1$$

$$W(n) = W(n-2) + 2$$

Level 2

$$W(n) = (W(n-2-1) + 1) + 2$$

$$W(n) = W(n-3) + 3$$

Generalized Equation

$$W(n) = W(n-k) + k$$

Recursion Depth

$$n - k = 0$$

$$k = n$$

Substitute

$$W(n) = W(n-n) + n$$

$$W(n) = W(0) + n$$

$$W(n) \in \Theta(n)$$

$$S(n) \in \Theta(n)$$

- **1d.**

$$W(n) = 2W(\frac{n}{2}) + 1$$

Level 1

$$W(n) = 2 \cdot (2W(\frac{n}{4}) + 1) + 1$$

$$W(n) = 4W(\frac{n}{4}) + 3$$

Level 2

$$W(n) = 4 \cdot (2W(\frac{n}{8}) + 1) + 3$$

$$W(n) = 8W(\frac{n}{8}) + 7$$

Generalized Equation

$$W(n) = 2^k W(\frac{n}{2^k}) + (2^k - 1)$$

Recursion Depth

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

$$k = \lg n$$

Substitute

$$W(n) = 2^{\lg n} W(\frac{n}{2^{\lg n}}) + (2^{\lg n} - 1)$$

$$W(n) = nW(1) + n - 1$$

$$W(n) \in \Theta(n)$$

$$S(n) = S(\frac{n}{2}) + 1$$

Level 1

$$S(n) = S(\frac{n}{4}) + 1 + 1$$

$$S(n) = S(\frac{n}{4}) + 2$$

Level 2

$$S(n) = S(\frac{n}{8}) + 1 + 2$$

$$S(n) = S(\frac{n}{8}) + 3$$

Generalized Equation

$$S(n) = S(\frac{n}{2^k}) + k$$

Recursion Depth

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

$$k = \lg n$$

Substitute

$$S(n) = S(\frac{n}{2^{\lg n}}) + \lg n$$

$$S(n) = S(1) + \lg n$$

$$\boxed{S(n) \in \Theta(\log n)}$$

• **1e.**

$$W(n) = W(\frac{n}{3}) + W(\frac{2n}{3}) + 1$$

Level 1

$$W(n) = [W(\frac{n}{9}) + W(\frac{2n}{9}) + 1] + [W(\frac{2n}{9}) + W(\frac{4n}{9}) + 1] + 1$$

$$W(n) = W(\frac{n}{9}) + 2W(\frac{2n}{9}) + W(\frac{4n}{9}) + 3$$

Observations

$$\frac{n}{9} + \frac{2n}{9} + \frac{2n}{9} + \frac{4n}{9} = n$$

Subproblem sizes sum to n.

Changes to parallelism do not affect total work. The ureduce algorithm performs the same overall computation as the reduce algorithm with different parallelism.

Both of these algorithms should therefore require the same work.

Because these algorithms are both asymptotically dominant when used in the implementation of rsearch, the work of both implementations should be the same.

$$\boxed{W(n) \in \Theta(n)}$$

$$S(n) = S(\frac{2n}{3}) + \Theta(1)$$

Level 1

$$S(n) = S(\frac{2}{3} \cdot \frac{2}{3} \cdot n) + 2\Theta(1)$$

Generalized Equation

$$S(n) = S((\frac{2}{3})^k n) + k\Theta(1)$$

Recursion Depth

$$(\frac{2}{3})^k n = 1$$

$$(\frac{2}{3})^k = \frac{1}{n}$$

$$k \log \frac{2}{3} = \log \frac{1}{n}$$

$$k \log \frac{2}{3} = -\log n$$

$$k = -\frac{\log n}{\log \frac{2}{3}}$$

$$k = \frac{\log n}{\log \frac{3}{2}}$$

$$k = \log_{3/2} n$$

Substitute

$$S(n) = S((\frac{2}{3})^{\log_{3/2} n} n) + \log_{3/2} n \Theta(1)$$

$$S(n) = S((\frac{3}{2})^{-1 \cdot \log_{3/2} n} n) + \log_{3/2} n \Theta(1)$$

$$S(n) = S(\frac{1}{(\frac{3}{2})^{\log_{3/2} n}} n) + \log_{3/2} n \Theta(1)$$

$$S(n) = S(1) + \log_{3/2} n \Theta(1)$$

$$\boxed{S(n) \in \Theta(\log n)}$$

• **2a.**

$$\text{dedup}(A) = \text{map}(\lambda i. A_i, \text{filter}(\lambda i. \neg \text{reduce}(\lambda x, j. x \vee (A_j = A_i), \text{False}, \langle 0, \dots, i-1 \rangle), \langle 0, \dots, n-1 \rangle)))$$

Work

Map handles a list of potentially n unique entries in the worst case: $W_M(n) \in \Theta(n)$

Filter processes a list of n entries: $W_F(n) \in \Theta(n)$

Reduce handles a list of i entries, with $i \leq n-1$ being the index of the list Filter is currently processing.

Work for one Reduce at index i : $W_R(i) = \Theta(i)$

Map runs once on the list filter returns. Filter runs a Reduce process for each entry in the list.

$$W_{\text{dedup}}(n) = W_F(n) + \sum_{i=0}^{n-1} W_R(i) + W_M(n)$$

$$W_{\text{dedup}}(n) = \Theta(n) + \Theta(n^2) + \Theta(n)$$

$$W_{\text{dedup}}(n) = \Theta(n^2) + 2\Theta(n)$$

$$W_{\text{dedup}}(n) \in \Theta(n^2)$$

Span

Map handles a list of potentially n unique entries in the worst case, each one in parallel with constant time:

$$S_M(n) \in \Theta(1)$$

Filter processes a list of n entries, in parallel with a binary tree: $S_F(n) \in \Theta(\log n)$

Reduce handles a list of i entries in parallel with a binary tree, with $i \leq n$ being the index of the list Filter is currently processing.

The maximum size for one Reduce at index i : $\max_{0 \leq i < n} S_R(i) = \max_{0 \leq i < n} \Theta(\log i) \in \Theta(\log n)$

Adding that together gives us:

$$S_{\text{dedup}}(n) = S_F(n) + \max_{0 \leq i < n} S_R(i) + S_M(n)$$

$$S_{\text{dedup}}(n) = \Theta(\log n) + \Theta(\log n) + \Theta(1)$$

$$S_{\text{dedup}}(n) = 2\Theta(\log n) + \Theta(1)$$

$$S_{\text{dedup}}(n) \in \Theta(\log n)$$

• **2b.**

$$\begin{aligned} \text{multi-dedup}(A) = & \text{let } B = \text{flatten}(A) \text{ in} \\ & \text{map} \left(\lambda i. B_i, \right. \\ & \quad \text{filter} \left(\lambda i. \neg \text{reduce}(\lambda x, j. x \vee (B_j = B_i), \right. \\ & \quad \quad \text{False}, \{0, \dots, i-1\}), \\ & \quad \quad \left. \left. \{0, \dots, N-1\} \right) \right) \end{aligned}$$

A is a list of lists indexed $A_0, \dots, A_m$. There are a total of $m+1$ lists in A .

n is the number of entries in each list. For this analysis the total number of entries in A is $N = (m+1)n$

Work

Flatten processes every entry in each list into one big list. $W_{\text{flatten}} = \Theta(N)$

Map handles a list of potentially N unique entries in the worst case: $W_M(N) \in \Theta(N)$

Filter processes a list of N entries: $W_{\text{filter}}(N) \in \Theta(N)$

Reduce handles the i preceding indices for the element currently being processed by Filter.

$$W_R(i) = \Theta(i)$$

$$W_{\text{multi-dedup}}(N) = W_{\text{flatten}}(N) + W_{\text{filter}}(N) + \sum_{i=0}^{N-1} W_R(i) + W_M(N)$$

$$W_{\text{multi-dedup}}(N) = \Theta(N) + \Theta(N) + \sum_{i=0}^{N-1} \Theta(i) + \Theta(N)$$

$$W_{\text{multi-dedup}}(N) = 3\Theta(N) + \Theta(N^2)$$

$$W_{\text{multi-dedup}}(N) \in \Theta(N^2)$$

$$W_{\text{multi-dedup}}((m, n) \in \Theta((m+1)^2 n^2))$$

Span

Flatten processes a list of N entries, in parallel with a binary tree: $S_{\text{flatten}}(N) \in \Theta(\log N)$

Map handles a list of potentially N unique entries in the worst case, each one in parallel with constant time: $S_M(N) \in \Theta(1)$

Filter processes a list of N entries, in parallel with a binary tree: $S_{filter}(N) \in \Theta(\log N)$

Reduce handles a list of i entries in parallel with a binary tree, where $(i \leq N)$ is the index currently being processed by filter.

$$S_R(i) \in \Theta(\log i)$$

$$\max_{0 \leq i < N} S_R(i) = \max_{0 \leq i < N} \Theta(\log i) \in \Theta(\log N)$$

Adding that together gives us:

$$S_{multi-dedup}(N) = S_{flatten}(N) + S_{filter}(N) + \max_{0 \leq i < N} S_R(i) + S_M(N)$$

$$S_{multi-dedup}(N) = \Theta(\log N) + \Theta(\log N) + \Theta(\log N) + \Theta(1)$$

$$S_{multi-dedup}(N) = 3\Theta(\log N) + \Theta(1)$$

$$S_{multi-dedup}(N) \in \Theta(\log N)$$

$$S_{multi-dedup}((m, n) \in \Theta(\log((m+1)n)))$$

• **2c.**

Sequence operations are useful in both dedup and multi-dedup because they allow for parallelism, which lowers span.

Lowering span lowers total execution time for the algorithm provided there are enough processors to accomodate it.

Parallelism P_a

$$P_a = \frac{W(n)}{S(n)}$$

Execution Time T_P , Processors P

$$T_P \geq \max\left(\frac{W(n)}{P}, S(n)\right)$$

When the number of processors P approaches available parallelism P_a , the execution time T_P approaches the span $S(n)$ so that $T_P \approx S(n)$. Adding more processors beyond this point will provide little improvement to execution time T_P .

• **3b.**

$$W(n) = W(n-1) + 1$$

Level 1

$$W(n) = (W(n-1-1) + 1) + 1$$

$$W(n) = W(n-2) + 2$$

Level 2

$$W(n) = (W(n-2-1) + 1) + 2$$

$$W(n) = W(n-3) + 3$$

Generalized Equation

$$W(n) = W(n-k) + k$$

Recursion Depth

$$n - k = 0$$

$$k = n$$

Substitute

$$W(n) = W(n-n) + n$$

$$W(n) = W(0) + n$$

$$W(n) \in \Theta(n)$$

$$S(n) = S(n-1) + 1$$

Level 1

$$S(n) = (S(n-1-1) + 1) + 1$$

$$S(n) = S(n-2) + 2$$

Level 2

$$S(n) = (S(n-2-1) + 1) + 2$$

$$S(n) = S(n-3) + 3$$

Generalized Equation

$$S(n) = S(n-k) + k$$

Recursion Depth

$$n - k = 0$$

$$k = n$$

Substitute

$$S(n) = S(n-n) + n$$

$$S(n) = S(0) + n$$

$$\boxed{S(n) \in \Theta(n)}$$

• **3d.**

Work

Scan (efficient contraction version): $W_s(n) = W_s(\frac{n}{2}) + \Theta(n)$. The work across all levels follows this pattern: $n(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots)$. $W_s(n) \in \Theta(n)$.

Map applies `paren_map` to all n entries; each application takes constant work. $W_m(n) = n\Theta(1)$. $W_m(n) \in \Theta(n)$

Reduce processes all entries in parallel using a binary tree. Work is split in half, but both halves are processed.

$$W_r(n) = 2W_r(\frac{n}{2}) + \Theta(1). \quad W_r(n) \in \Theta(n)$$

$$W_{\text{parens_match_scan}}(n) = W_s(n) + W_m(n) + W_r(n) + \Theta(1)$$

$$W_{\text{parens_match_scan}}(n) = \Theta(n) + \Theta(n) + \Theta(n) + \Theta(1)$$

$$W_{\text{parens_match_scan}}(n) = 3\Theta(n) + \Theta(1)$$

$$\boxed{W_{\text{parens_match_scan}}(n) \in \Theta(n)}$$

Span

Scan (efficient contraction version): $S_s(n) = S_s(\frac{n}{2}) + \Theta(1)$. There are $\log_2 n$ recursion levels. $S_s(n) \in \Theta(\log n)$

Map applies `paren_map` to all n entries in parallel and each application takes constant work. $S_m(n) \in \Theta(1)$

Reduce processes all entries in parallel using a binary tree. There are $\log_2 n$ recursion levels. $S_r(n) \in \Theta(\log n)$

$$S_{\text{parens_match_scan}}(n) = S_s(n) + S_m(n) + S_r(n) + \Theta(1)$$

$$S_{\text{parens_match_scan}}(n) = \Theta(\log n) + \Theta(1) + \Theta(\log n) + \Theta(1)$$

$$S_{\text{parens_match_scan}}(n) = 2\Theta(\log n) + 2\Theta(1)$$

$$\boxed{S_{\text{parens_match_scan}}(n) \in \Theta(\log n)}$$

• **3f.**

$$W(n) = 2W(\frac{n}{2}) + 1$$

Level 1

$$W(n) = 2 \cdot (2W(\frac{n}{4}) + 1) + 1$$

$$W(n) = 4W(\frac{n}{4}) + 3$$

Level 2

$$W(n) = 4 \cdot (2W(\frac{n}{8}) + 1) + 3$$

$$W(n) = 8W(\frac{n}{8}) + 7$$

Generalized Equation

$$W(n) = 2^k W(\frac{n}{2^k}) + (2^k - 1)$$

Recursion Depth

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

$$k = \lg n$$

Substitute

$$W(n) = 2^{\lg n} W(\frac{n}{2^{\lg n}}) + (2^{\lg n} - 1)$$

$$W(n) = n \cdot W(1) + (n - 1)$$

$$\boxed{W(n) \in \Theta(n)}$$

$$S(n) = S(\frac{n}{2}) + 1$$

Level 1

$$S(n) = (S(\frac{n}{4}) + 1) + 1$$

$$S(n) = S(\frac{n}{4}) + 2$$

Level 2

$$S(n) = (S(\frac{n}{8}) + 1) + 2$$

$$S(n) = S(\frac{n}{8}) + 3$$

Generalized Equation $S(n) = S(\frac{n}{2^k}) + k$

Recursion Depth

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

$$k = \lg n$$

Substitute

$$S(n) = S(\frac{n}{2^{\lg n}}) + \lg n$$

$$S(n) = S(1) + \lg n$$

$$\boxed{S(n) \in \Theta(\lg n)}$$